

Pilhas

Algoritmos e Programação 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

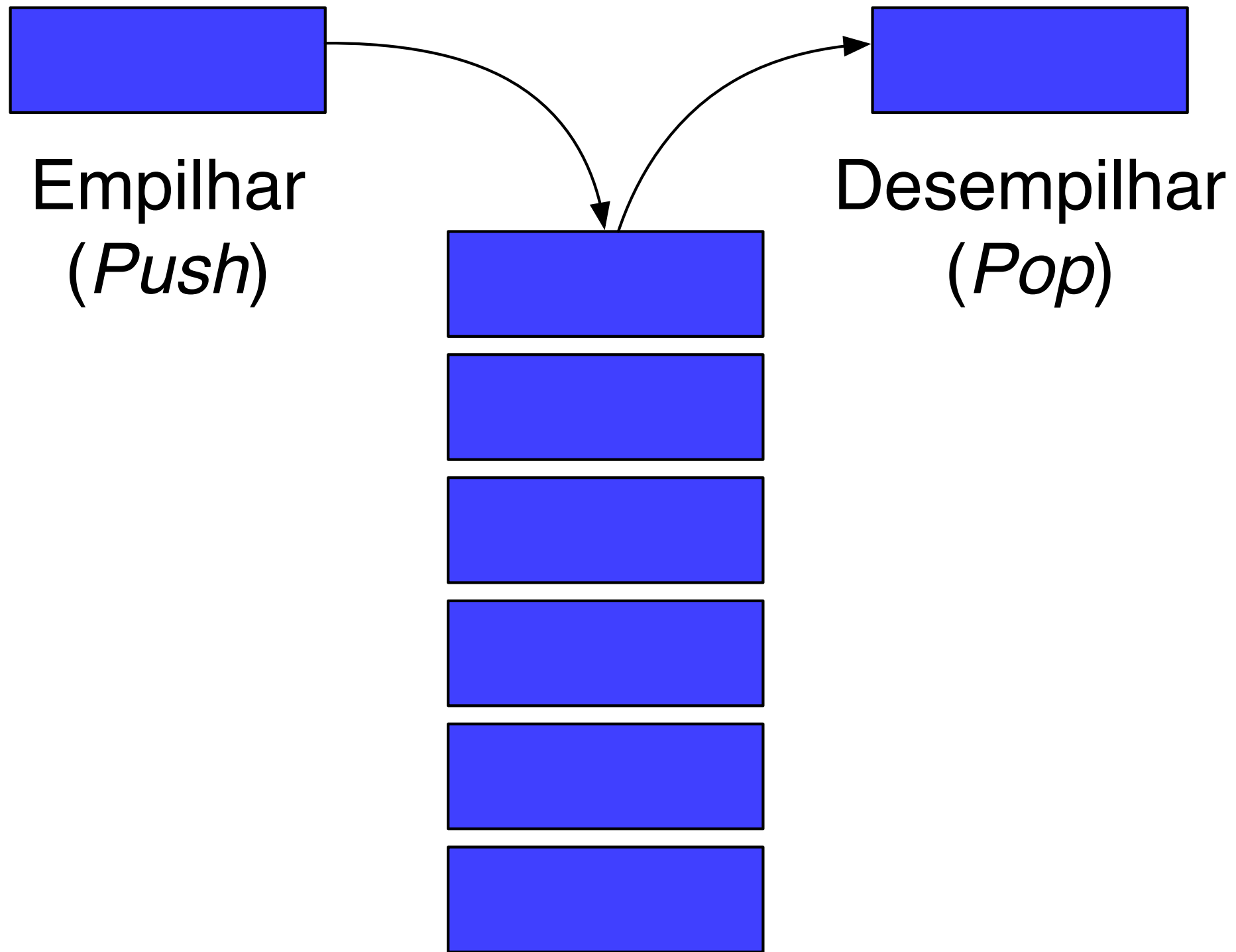
Introdução

- **Pilhas** e **filas** são conjuntos dinâmicos que seguem uma política pré-especificada para inserção e remoção de elementos
- São um **Tipo Abstrato de Dados (TAD)**, em inglês ADT – *Abstract Data Type*
 - Descrição lógica
- vs **estrutura de dados** que são representações concretas em nível de implementação

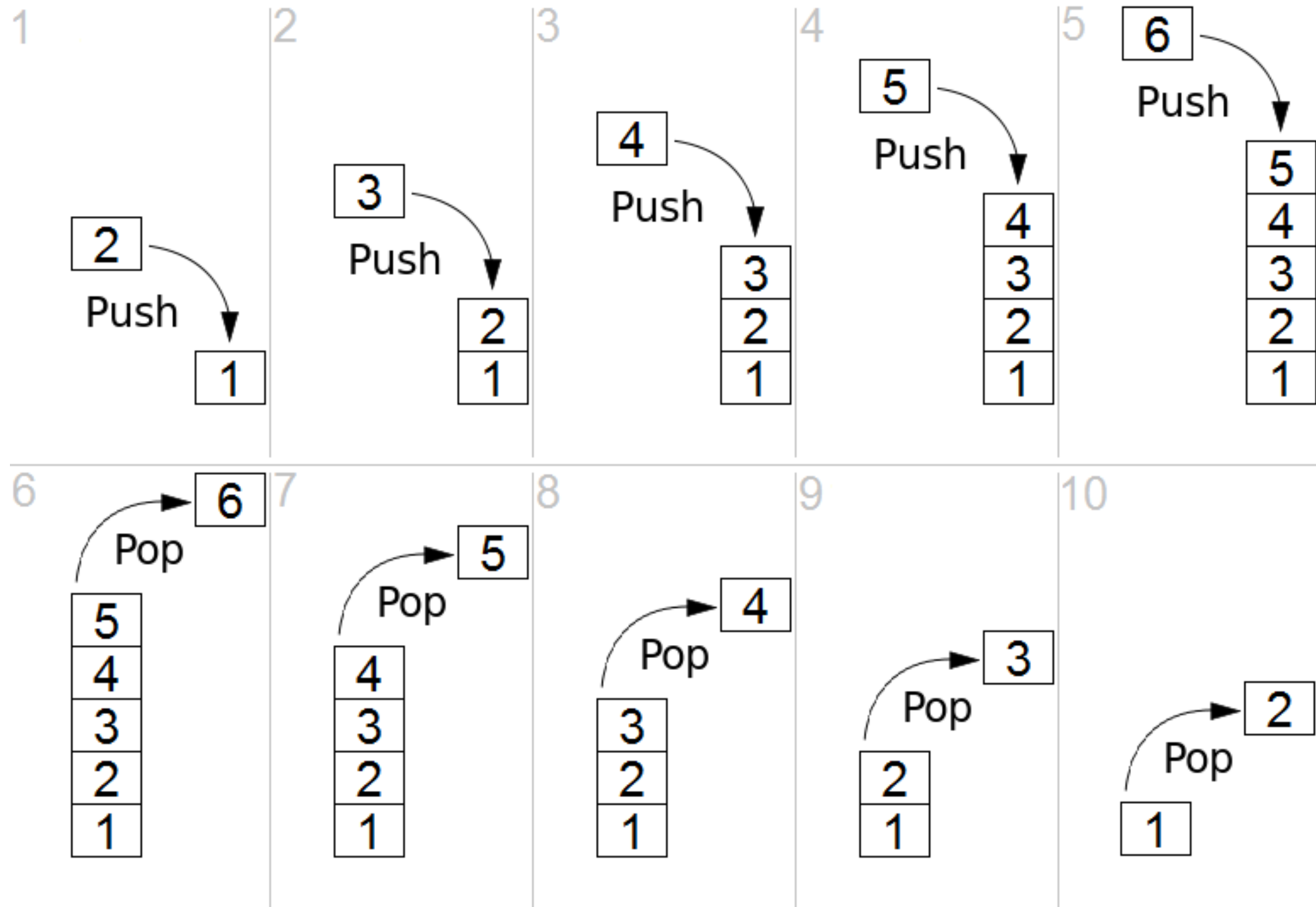
Pilha de Livros



Pilha TAD



Sequência de Operações



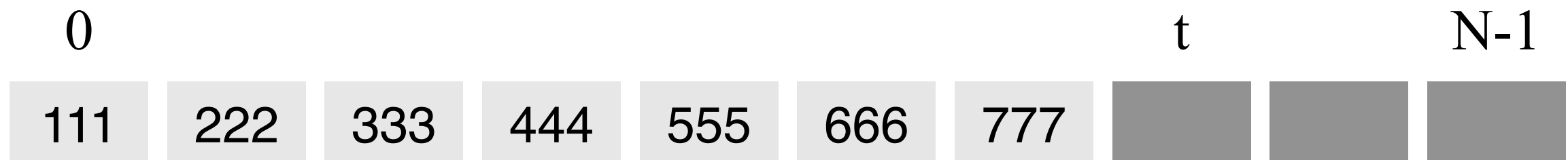
Definição

- **Pilha** é um **TAD** que serve como uma coleção de elementos, com duas operações principais:
 - **empilhar** (*push*) adiciona um elemento para coleção;
 - **desempilhar** (*pop*) remove o elemento mais recentemente adicionado;
- A política de inserção e remoção dos elementos é conhecida pelo acrônimo **LIFO** (*Last In, First Out*), em português **Último a Entrar, Primeiro a Sair**

Como Implementar?

- **TAD** não diz como implementar
- Existem diversas maneiras eficientes de implementar pilhas. Mostraremos como implementar utilizando **arranjos simples**, entretanto podemos implementá-las por meio de **listas encadeadas**

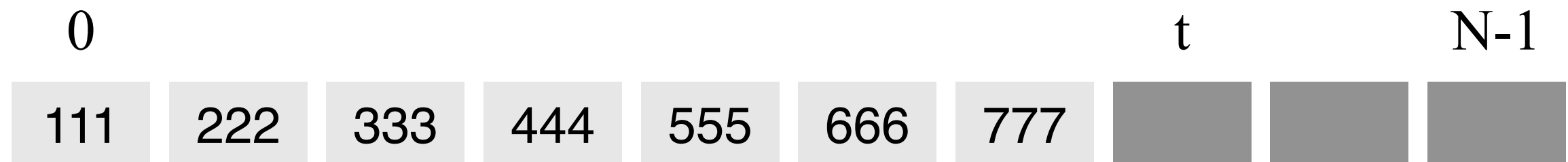
Implementação em Vetor



O vetor $p[0..t - 1]$ armazena uma pilha.

- Suponha que nossa pilha está armazenada em um vetor $p[0..N - 1]$
- A parte do vetor efetivamente ocupada pela pilha é $p[0..t - 1]$
- Índice $t - 1$ define o **topo** da pilha

Implementação em Vetor (Cont.)



O vetor $p[0..t - 1]$ armazena uma pilha.

- Pilha está **vazia** se t vale 0
- Pilha está **cheia** se t vale N

Overflow e Underflow

- **Overflow** (*transbordamento*): Ocorre quando se tenta inserir um elemento em uma estrutura que já atingiu a sua capacidade máxima
- **Underflow** (*esvaziamento*): Ocorre quando se tenta remover um elemento em uma estrutura que não possui elementos
- Por questões de legibilidade, os códigos desta aula omitem o tratamento dessas situações

Inicializar

- Inicialmente devemos sinalizar que a pilha está vazia, portanto iremos inicializar $t = 0$

Desempilhar (*Remove*)

- Para **desempilhar** um elemento da pilha, ou seja, para remover um elemento, faça:

```
t = t - 1;  
x = p[t];
```

- É claro que o programador não deve fazer isto se a pilha estiver vazia, caso contrário ocorrerá *esvaziamento da pilha* (*stack underflow*)

Topo (*Consultar*)

- Para **consultar** a pilha sem desempilhar basta fazer:

```
x = p[t - 1];
```

Empilhar (*Inserir*)

- Para **empilhar** um objeto y , ou seja, para inserir y na pilha faça:

```
p[t] = y;  
t = t + 1;
```

- Antes de empilhar, é preciso ter certeza de que a pilha não está cheia (*stack overflow*)
 - A tentativa de inserir em uma pilha cheia é um indício de mau planejamento lógico do seu programa

Algoritmo: Empilhar

```
1. void empilhe(int p[], int &t, int x) {  
2.     p[t] = x;  
3.     t = t + 1;  
4. }
```


Algoritmo: Desempilhar

```
1. int desempilhe(int p[], int &t) {  
2.     t = t - 1;  
3.     return p[t];  
4. }
```


Aplicação: Notação Posfixa

- Expressões aritméticas são usualmente escritas em notação *infixa*
 - Operadores ficam entre os operandos
- Na notação *posfixa* (ou *polonesa*) os operadores ficam depois dos operandos

Notação Infixa e Posfixa

Notação infixa

$$(A + B * C)$$

$$(A * (B + C) / D - E)$$

$$(A + B * (C - D * (E - F) - G * H) - I * 3)$$

$$(A + B * C / D * E - F)$$

$$(A * (B + (C * (D + (E * (F + G)))))))$$

Notação posfixa

$$A B C * +$$

$$A B C + * D / E -$$

$$A B C D E F - * - G H * - * + I 3 * -$$

$$A B C * D / E * + F -$$

$$A B C D E F G + * + * + *$$

Expressões aritméticas em notação infixa e notação posfixa. A notação posfixa dispensa parênteses. Os operandos (A, B etc.) aparecem na mesma ordem nas duas notações.

Problema

- Traduzir para notação posfixa uma expressão infixada dada
- Para simplificar, suporemos que:
 - A expressão infixada está correta
 - Cada nome de variável tem uma letra apenas
 - A expressão toda está embrulhada em um par de parênteses

Ideia

1. Escaneie a expressão infixa da esquerda para a direita.
2. Se o caractere escaneado for um operando, coloque-o na expressão posfixa.
3. Caso contrário:
 - a) Se precedência do operador atual for maior que precedência do operador no topo da pilha, ou se pilha está vazia, ou se a pilha contiver um '(', então empilhe o operador atual
 - b) Caso contrário, desempilhe todos os operadores da pilha que tenham precedência maior ou igual à do operador atual. Depois disso, empilhe o operador atual para a pilha
4. Se caractere escaneado for um '(', empilhe-o;
5. Se o caractere escaneado for um ')', desempilhe até que um '(' seja encontrado, e descarte ambos os parênteses.
6. Repita as etapas 2 a 5 até que a expressão infixa seja escaneada.
7. Quando a varredura terminar, desempilhe da pilha e adicione os operadores na expressão posfixa até que ela não esteja vazia.
8. Finalmente, imprima a expressão posfixa.

Algoritmo Infixa para Posfixa

```
1. char infix_para_posfixa(char infix[]) {  
2.     /* PARTE 1 */  
3.     char *posfix, x;  
4.     char *p; int t;  
5.     int n, i, j;  
6.     n = strlen(infix);  
7.     posfix = (char *) malloc (n * sizeof(char));  
8.     p = (char *) malloc(n * sizeof(char));  
9.     t = 0; p[t++] = infix[0];  
10.    /* PARTE 2 */  
11.    ...  
12.}
```

```
1. char infixa_para_posfixa(char infix[]) {
2.     /* PARTE 1 */
3.     ...
4.     /* PARTE 2 */
5.     for (j = 0, i = 1; infix[i] != '\0'; i++) {
6.         switch(infix[i]) {
7.             case '(': p[t] = infix[i]; t = t + 1; /* Empilhe */
8.             break;
9.             case ')': while (1) {
10.                 t = t - 1; x = p[t]; /* Desempilhe */
11.                 if (x == '(') break;
12.                 posfix[j] = x; j = j + 1; }
13.             break;
14.             case '+':
15.             case '-': while (1) {
16.                 x = p[t-1]; /* Topo */
17.                 if (x == '(') break;
18.                 t = t - 1;
19.                 posfix[j] = x; j = j + 1; }
20.             p[t] = infix[i]; t = t + 1; /* Empilhe */
21.             break;
22.             case '*':
23.             case '/': while (1) {
```

```

13.         break;
14.     case '+':
15.     case '-': while (1) {
16.         x = p[t-1]; /* Topo */
17.         if (x == '(') break;
18.         t = t - 1;
19.         postfix[j] = x; j = j + 1; }
20.         p[t] = infix[i]; t = t + 1; /* Empilhe */
21.     break;
22.     case '*':
23.     case '/': while (1) {
24.         x = p[t-1]; /* Topo */
25.         if (x == '(' || x == '+' || x == '-')
26.             break;
27.         t = t - 1;
28.         postfix[j] = x; j = j + 1; }
29.         p[t] = infix[i]; t = t + 1; /* Empilhe */
30.     break;
31.     default: postfix[j] = infix[i]; j = j + 1; }
32. }
33. free(p);
34. postfix[j] = '\0';
35. return postfix;
36. }

```


Exemplo Infixa para Posfixa

infix[0..i-1]	p[0..t-1]	posfix[0..j-1]
(	(	
(A	(	A
(A *	(*	A
(A * (	(* (	A
(A * (B	(* (	A B
(A * (B	(* (	A B
(A * (B *	(* (*	A B
(A * (B * C	(* (*	A B C
(A * (B * C +	(* (+	A B C *
(A * (B * C + D	(* (+	A B C * D
(A * (B * C + D)	(*	A B C * D +
(A * (B * C + D))		A B C * D + *

Resultado da aplicação da função *infixa_para_posfixa*.

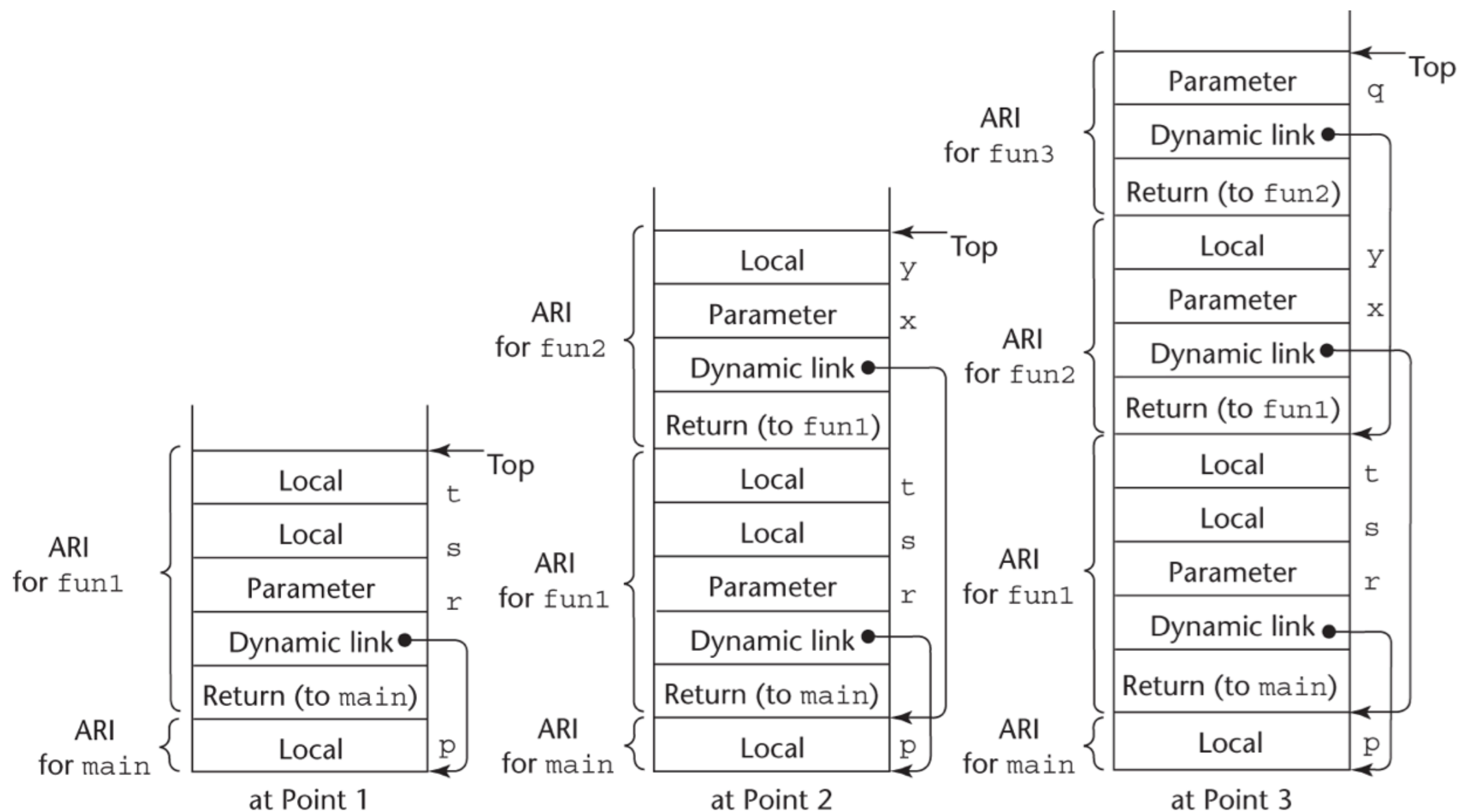
A Pilha de Execução de um Programa

- Todo programa em C/C++ é composto por uma ou mais funções, sendo **main** a primeira função a ser executada
- Para executar um programa, o computador usa uma “**pilha de execução**”

Pilha de Execução

- Ao encontrar a invocação de uma função, o computador cria um novo “espaço de trabalho”, que contém todos os parâmetros e todas as variáveis locais da função
- Esse espaço de trabalho é colocado na **pilha de execução** (sobre o espaço de trabalho que invocou a função) e a execução da função começa (confinada ao seu espaço de trabalho)

Registro de Ativação e Pilha de Execução



ARI = activation record instance

Pilha de Execução (Cont.)

- Quando execução da função termina, o seu espaço de trabalho é retirado da pilha e descartado
- O espaço de trabalho que estiver agora no topo da pilha é reativado e a execução é retomada do ponto em que havia sido interrompida

Exemplo Pilha de Execução

```
1. int g(int a, int b) {  
2.     return a + b;  
3. }  
4.  
5. int f(int i, int j, int k) {  
6.     int x;  
7.     x = /* 2 */ g(i, j);  
8.     return x + k;  
9. }  
10.  
11. int main (void) {  
12.     int i, j, k, y;  
13.  
14.     i = 111; j = 222; k = 444;  
15.     y = /* 1 */ f(i, j, k);  
16.     printf("%d\n", y);  
17.     return 0;  
18. }
```

Execução Programa

O programa é executado da seguinte maneira:

1. Um espaço de trabalho é criado para a função **main**;
2. No ponto 1, a execução de **main** é suspensa e um espaço de trabalho para a função **f** é colocada na pilha;
3. No ponto 2, a execução da **f** é suspensa e um espaço de trabalho para a função **g** é colocado na pilha;
4. Quando a execução da **g** termina, a função devolve 333;
5. Quando a execução de **f** termina, a função devolve 777;

Execução Programa (Cont.)

- No nosso exemplo, **f** e **g** são funções distintas
 - Mas tudo funcionaria da mesma maneira se **f** e **g** fossem idênticas, ou seja, se **f** fosse uma função recursiva

Referências

- CORMEN, T. H. [et al]. Algoritmos: teoria e prática. 3a ed. Rio de Janeiro: Elsevier, 2012.
- FEOFILOFF, P. Algoritmos em Linguagem C, 1. ed. Rio de Janeiro: Elsevier, 2008.
- TENENBAUM, Aaron M; ANGENTEIN, Moshe; LANGSAM, Yedidiah. Estruturas de dados usando C. Sao Paulo, SP: Pearson, 1995. 884p.